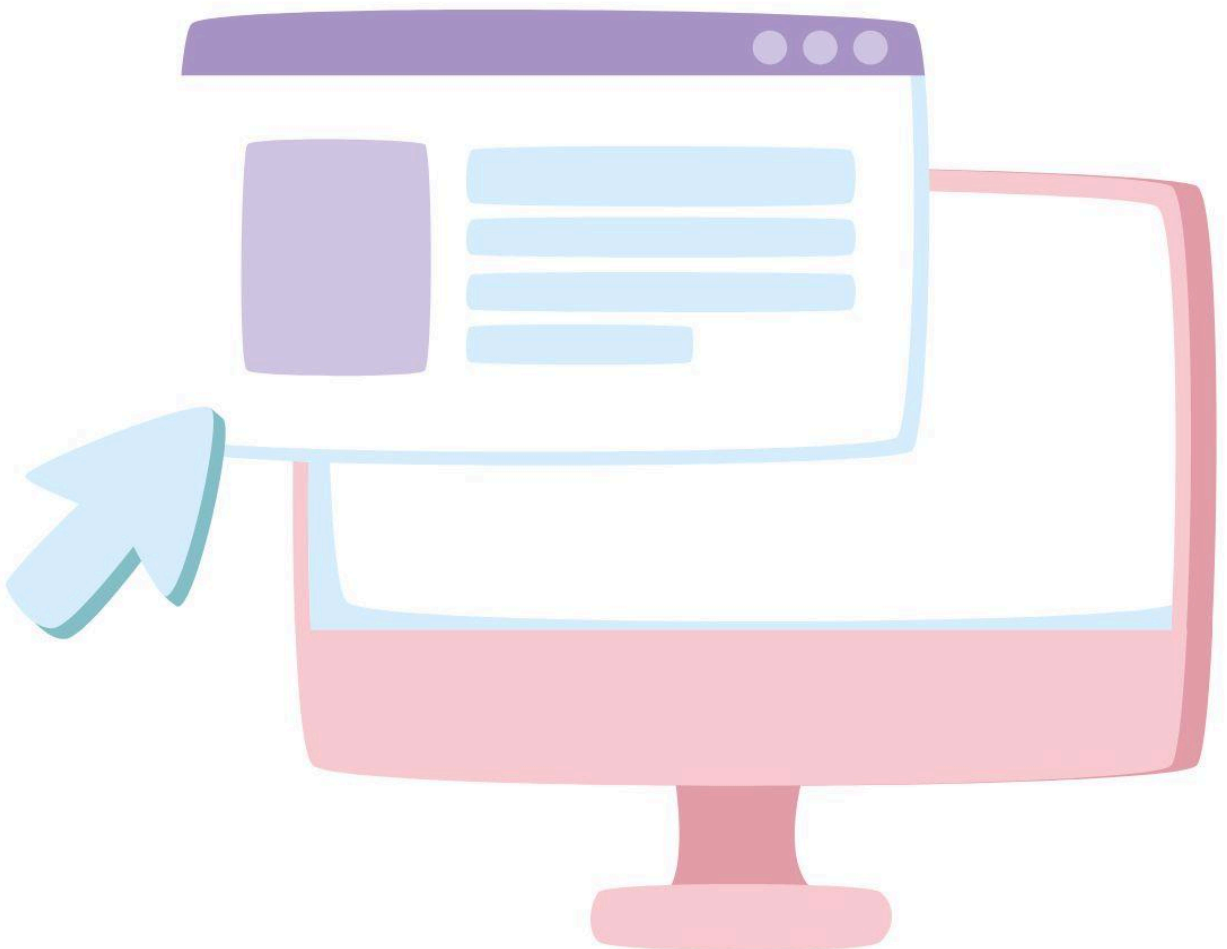


A1.1

Conceptos iniciales



ÍNDICE

Qué es un IDE y di al menos 3 de los más usados.	3
Ciclo de vida de un software.	3
Fases en el desarrollo y ejecución del software.	3
Diferencia entre código fuente, código objeto y código ejecutable.	4
Diferencia entre algoritmo, pseudocódigo y código.	5
¿Qué es el diseño estructurado y cuáles son los 3 tipos de construcciones en las que se basa?	5
Qué es un lenguaje de programación y clasificación de los lenguajes en función de su nivel de abstracción y según la forma de ejecución. Pon ejemplos.	6
• Clasificación según la forma de ejecución	6
Diferencia entre compilador e intérprete.	7
¿Qué es un bytecode?	7
En la programación orientada a objetos (POO), existen tres conceptos fundamentales: clase, objeto y método. Defínelos y pon un ejemplo práctico en un lenguaje que conozcas.	7
Ejemplo práctico:	7
URL REPOSITORIO: https://github.com/shooneyx/ETS-T1.git	8

Qué es un IDE y di al menos 3 de los más usados.

Un entorno de desarrollo integrado (IDE) es una aplicación de software que ayuda a los programadores a desarrollar código de software de manera eficiente. Aumenta la productividad de los desarrolladores al combinar capacidades como editar, crear, probar y empaquetar software en una aplicación fácil de usar. Así como los escritores utilizan editores de texto y los contables, hojas de cálculo, los desarrolladores de software utilizan IDE para facilitar su trabajo.

Los IDE más usados son:

- Visual Studio Code
- PyCharm
- IntelliJ IDEA
- Eclipse
- Xcode

Ciclo de vida de un software.

El ciclo de vida del desarrollo del software (SDLC) es el conjunto de fases que aseguran que el software cumpla los requisitos y se desarrolle con métodos adecuados. Incluye etapas como análisis, diseño, implementación, pruebas y mantenimiento.

Fases en el desarrollo y ejecución del software.

La metodología para el desarrollo de software son los procesos a seguir sistemáticamente para idear, implementar y mantener un producto de software desde que surge la necesidad del producto hasta que se cumple el objetivo por el cual fue creado.

Las distintas etapas del desarrollo de software son:

- Planificación: Se definen los objetivos, recursos, plazos y el alcance del proyecto.
- Análisis: Se estudian las necesidades del usuario y se documentan las funcionalidades que debe cumplir el sistema.
- Diseño: Se planifica la estructura interna del software (arquitectura, base de datos, interfaces, etc.).
- Implementación: Los programadores escriben el código fuente siguiendo el diseño.
- Pruebas: Se verifica que el software funcione correctamente y cumpla con los requisitos.
- Instalación o despliegue: El sistema se instala en el entorno real de uso.

- Uso y mantenimiento: Se corrigen errores, se actualiza y se mejora el software a lo largo del tiempo.

Las etapas de ejecución del software:

- Carga: el programa se carga desde el almacenamiento a la memoria principal.
- Ejecución: el procesador interpreta las instrucciones del programa en orden.
- Salida de resultados: el sistema muestra o guarda los resultados obtenidos.

Diferencia entre código fuente, código objeto y código ejecutable.

Para explicar mejor la diferencia entre los tres códigos, hice una tabla:

Tipo de código	Definición	Características principales	Extensión o ejemplo	Quién lo usa
Código Fuente	Texto escrito por el programador en un lenguaje de programación	→ Legible por humanos → Contiene instrucciones, estructuras y comentarios. → Necesita ser traducido para ejecutarse.	.c, .java, .py	Lo escribe el programador
Código Objeto	Resultado intermedio que produce el compilador al traducir el código fuente.	→ En lenguaje máquina parcial (binario). → No es ejecutable todavía. → Puede contener referencias a otras funciones o librerías.	.o, .obj	Lo genera el compilador
Código Ejecutable	Archivo final listo para ejecutarse en el sistema operativo.	→ En lenguaje máquina completo. → Todos los símbolos están resueltos. → Se puede ejecutar directamente.	.exe, .out, programa	Lo genera el enlazador (linker) y lo usa el SO.

Diferencia entre algoritmo, pseudocódigo y código.

Un algoritmo es un conjunto ordenado y finito de pasos que permiten resolver un problema o realizar una tarea determinada. Se expresa normalmente en lenguaje natural, en forma de lista de pasos o mediante diagramas de flujo.

Los algoritmos deben ser claros, precisos, finitos y tener un propósito definido. No dependen de ningún lenguaje de programación, sino que representan la idea o el método lógico que se va a seguir para llegar a una solución. (Ej: sumar los números)

El pseudocódigo es una forma de representar un algoritmo utilizando una estructura parecida a la de un lenguaje de programación, pero sin seguir las reglas exactas de ninguno. Se usa como paso intermedio entre el algoritmo y el código, ya que permite organizar las instrucciones de forma más clara, estructurada y entendible para los programadores.

→ Aunque se parece al código, no puede ser ejecutado por un ordenador, porque no pertenece a ningún lenguaje real.

El código es la implementación del pseudocódigo en un lenguaje de programación concreto (ej: JavaScript). Este código sigue una sintaxis estricta, ya que el ordenador solo entiende instrucciones escritas en un formato que su compilador o intérprete pueda procesar.

→ Es el paso final: el programa se puede ejecutar, probar y producir resultados reales.

¿Qué es el diseño estructurado y cuáles son los 3 tipos de construcciones en las que se basa?

El diseño estructurado es una metodología de programación que consiste en dividir un programa en partes o módulos más pequeños, fáciles de entender, probar y mantener. Su objetivo principal es mejorar la claridad, reducir errores y facilitar el desarrollo del software siguiendo una lógica ordenada.

Los tres tipos de construcciones básicas en las que se basa son:

1. **Secuencia:** Ejecución ordenada de instrucciones, una detrás de otra.
2. **Selección o decisión:** Permite tomar decisiones y ejecutar distintas acciones según una condición.
3. **Iteración o repetición:** Repite un bloque de instrucciones mientras se cumpla una condición.

Qué es un lenguaje de programación y clasificación de los lenguajes en función de su nivel de abstracción y según la forma de ejecución. Pon ejemplos.

Un lenguaje de programación es un conjunto de símbolos, reglas y palabras clave que permiten al programador escribir instrucciones para que un ordenador ejecute tareas específicas.

Sirve como medio de comunicación entre el ser humano y la máquina, permitiendo desarrollar programas, aplicaciones y sistemas informáticos.

- Clasificación según su nivel de atracción:
 - **Lenguajes de bajo nivel:** Están más cerca del lenguaje máquina y son difíciles de entender para las personas.
 - Requieren conocer el hardware del sistema.
 - Se usa en programación de sistemas o controladores.
 - Ejemplo: Lenguaje ensamblador (Assembly).
 - **Lenguajes de nivel medio:** Combinan características de bajo y alto nivel.
 - Permiten manipular hardware, pero con una sintaxis más comprensible.
 - Ejemplo: C
 - **Lenguajes de alto nivel:** Son más cercanos al lenguaje humano y fáciles de aprender.
 - Permiten centrarse en la lógica y no en el hardware.
 - Ejemplos: Python, JavaScript, C++.
- Clasificación según la forma de ejecución
 - **Lenguajes compilados:** El código fuente se traduce completamente a lenguaje máquina antes de ejecutarse.
 - Más rápidos al ejecutarse.
 - Ejemplos: C, C++.
 - **Lenguajes interpretados:** El código se ejecuta línea por línea mediante un intérprete.
 - Más lentos, pero más fáciles de depurar.
 - Ejemplos: Python, JavaScript.
 - **Lenguajes híbridos (compilados a bytecode):** Se compilan a un código intermedio (bytecode) que se ejecuta en una máquina virtual.
 - Combinan portabilidad y eficiencia.
 - Ejemplos: **Java (JVM), C# (.NET).**

Diferencia entre compilador e intérprete.

Un **compilador** es un programa que traduce todo el código fuente de un programa a lenguaje máquina antes de ejecutarlo. El resultado de esta traducción es un archivo

ejecutable que puede funcionar de manera independiente, sin necesidad del código original. Este proceso permite que el programa se ejecute más rápido, aunque la compilación inicial puede llevar más tiempo. Si el compilador encuentra errores, los muestra una vez ha analizado todo el código. Lenguajes como C, C++ o Go utilizan este método.

Un **intérprete** traduce y ejecuta el código fuente línea por línea, sin generar un archivo ejecutable. Esto significa que el programa se va ejecutando al mismo tiempo que se traduce. Si aparece un error, la ejecución se detiene en ese punto, lo que facilita la depuración. Sin embargo, la ejecución suele ser más lenta porque la traducción ocurre en tiempo real. Lenguajes como Python, JavaScript o PHP son interpretados.

¿Qué es un bytecode?

El bytecode es un código intermedio entre el código fuente y el código máquina.

Se genera cuando un programa escrito en un lenguaje de programación (como Java o Python) es compilado, pero no directamente a lenguaje máquina, sino a una forma más simple y portátil que puede ejecutarse en diferentes sistemas operativos.

Este bytecode no se ejecuta directamente por el hardware, sino por una máquina virtual (como la Java Virtual Machine), que se encarga de interpretarlo o compilarlo en tiempo real.

En la programación orientada a objetos (POO), existen tres conceptos fundamentales: clase, objeto y método.

Defínelos y pon un ejemplo práctico en un lenguaje que conozcas.

- **Clase:** Una clase es un modelo o plantilla que define las características y comportamientos que tendrán los objetos. Dentro de una clase se definen los atributos (propiedades o datos) y los métodos (acciones o funciones) que podrán usar los objetos creados a partir de ella.
- **Objeto:** Un objeto es una instancia concreta de una clase, es decir, una realización específica con valores propios. Cada objeto tiene sus propios datos (atributos) y puede ejecutar los métodos definidos en la clase.
- **Método:** Un método es una función dentro de una clase que describe el comportamiento o acción que puede realizar un objeto.

Ejemplo práctico:

```
// Definición de la clase
class Animal {
    constructor(nombre, tipo, edad) {
        this.nombre = nombre;
    }
}
```

```

    this.tipo = tipo;
    this.edad = edad;
}

comer() {
    console.log(`${this.nombre} está comiendo.`);
}

dormir() {
    console.log(`${this.nombre} está durmiendo.`);
}
}

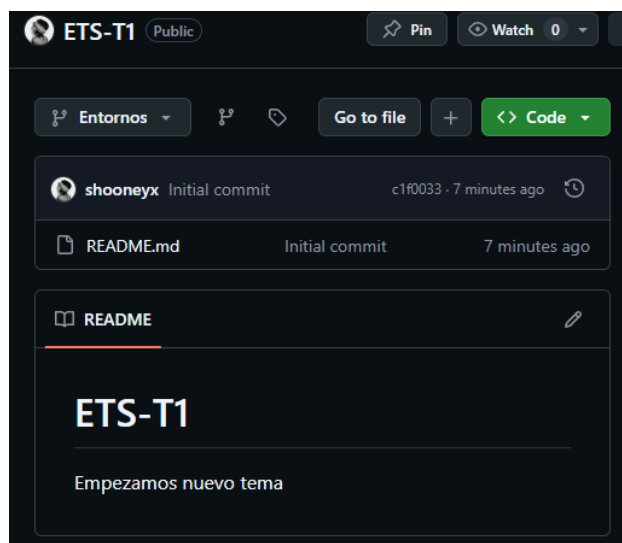
// Creación de objetos
let tigre = new Animal("Raja", "Tigre", 5);
let mono = new Animal("Chico", "Mono", 3);
let jirafa = new Animal("Luna", "Jirafa", 7);

// Uso de métodos
tigre.comer();
mono.dormir();
jirafa.comer();

```

Pd: Se debe crear una rama en github llamada `conceptosInicialesTuapellido`

URL REPOSITORIO: <https://github.com/shooneyx/ETS-T1.git>




```
AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (main)
● $ git branch
  * main
```

```
AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (main)
● $ git branch conceptosInicialesCerqueiras
```

```
AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (main)
● $ git branch
    conceptosInicialesCerqueiras
  * main
```

```
AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (main)
$ git checkout conceptosInicialesCerqueiras
● D      PruebaGit/indeex.html
  D      PruebaGit/prueba.js
  D      PruebaGit/style.css
Switched to branch 'conceptosInicialesCerqueiras'

AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
○ $
```

```
AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
● $ git remote -v
origin https://github.com/shooneyx/ETS.git (fetch)
origin https://github.com/shooneyx/ETS.git (push)

AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
$ git config --global user.name "Shooneyx"
● git config --global user.email "sheila.twocc@gmail.com"

AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
● $ git remote remove origin

AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
● $ git remote -v

AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
● $ git remote add origin https://github.com/shooneyx/ETS-T1.git
```

```
AlumGS@DESKTOP-CBN6BNT MINGW64 /f/ETS (conceptosInicialesCerqueiras)
$ git push -u origin conceptosInicialesCerqueiras
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.63 KiB | 1.63 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'conceptosInicialesCerqueiras' on
remote: GitHub by visiting:
remote:   https://github.com/shooneyx/ETS-T1/pull/new/conceptosInicialesCerqueiras
remote:
To https://github.com/shooneyx/ETS-T1.git
 * [new branch]      conceptosInicialesCerqueiras -> conceptosInicialesCerqueiras
branch 'conceptosInicialesCerqueiras' set up to track 'origin/conceptosInicialesCerqueiras'.
```

ETS-T1 Public

conceptosInicialesCerqueiras had recent pushes 18 seconds ago [Compare & pull request](#)

Entornos [Go to file](#) [Code](#)

Switch branches/tags

Find or create a branch...

Branches **Tags**

✓ Entornos default

conceptosInicialesCerqueiras

[View all branches](#)

Empezamos nuevo tema